

Quiz 1 Rubric

Automata Theory Monsoon 2026, IIIT Hyderabad

September 7, 2026

1. [3 points] Let $\Sigma = \{a, b, c\}$ and

$$L = \{w \in \Sigma^* \mid w \text{ does not contain } ab \text{ as a substring}\}.$$

- a. [1 point] Construct a DFA that recognizes L .
b. [2 points] Construct a regular expression r such that $L(r) = L$.

Solution:

Any submission that overlooked that c and carried out the construction (DFA and regex) only for a and b , has been awarded 0 for the whole question.

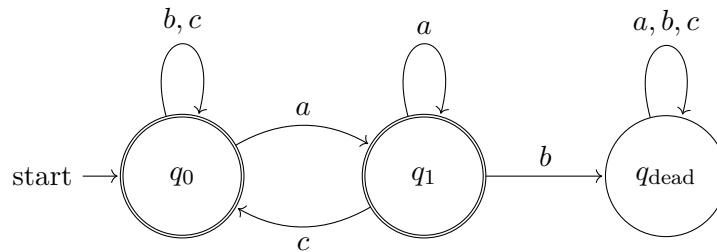
Part (a): DFA for L

[1 pt]

1. Minimal

$$M = (Q, \Sigma, \delta, q_0, F), \quad Q = \{q_0, q_1, q_{\text{dead}}\}, \quad F = \{q_0, q_1\}$$

δ	a	b	c
$\rightarrow q_0^*$	q_1	q_0	q_0
q_1^*	q_1	q_{dead}	q_0
q_{dead}	q_{dead}	q_{dead}	q_{dead}

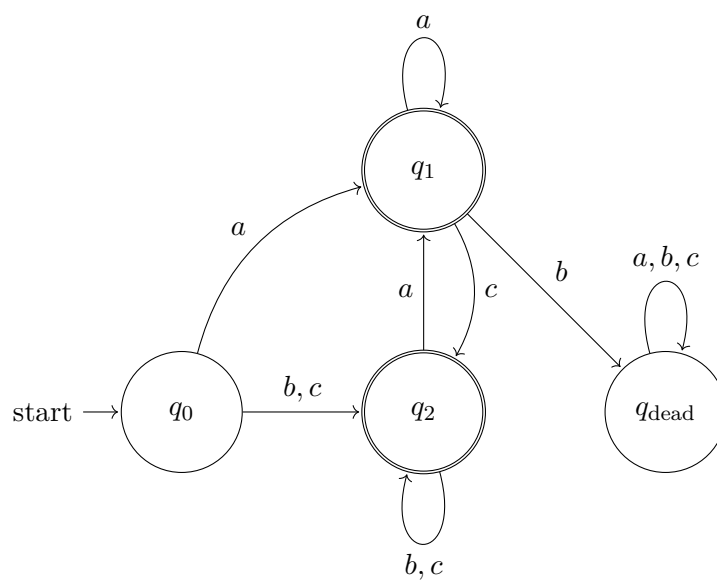


2. Non-minimal variant

q_0 is a non-accepting state and an added accepting state q_2 absorbs the b, c traffic that would otherwise loop on q_0 .

$$Q = \{q_0, q_1, q_2, q_{\text{dead}}\}, \quad F = \{q_1, q_2\}$$

δ	a	b	c
$\rightarrow q_0$	q_1	q_2	q_2
q_1^*	q_1	q_{dead}	q_2
q_2^*	q_1	q_2	q_2
q_{dead}	q_{dead}	q_{dead}	q_{dead}



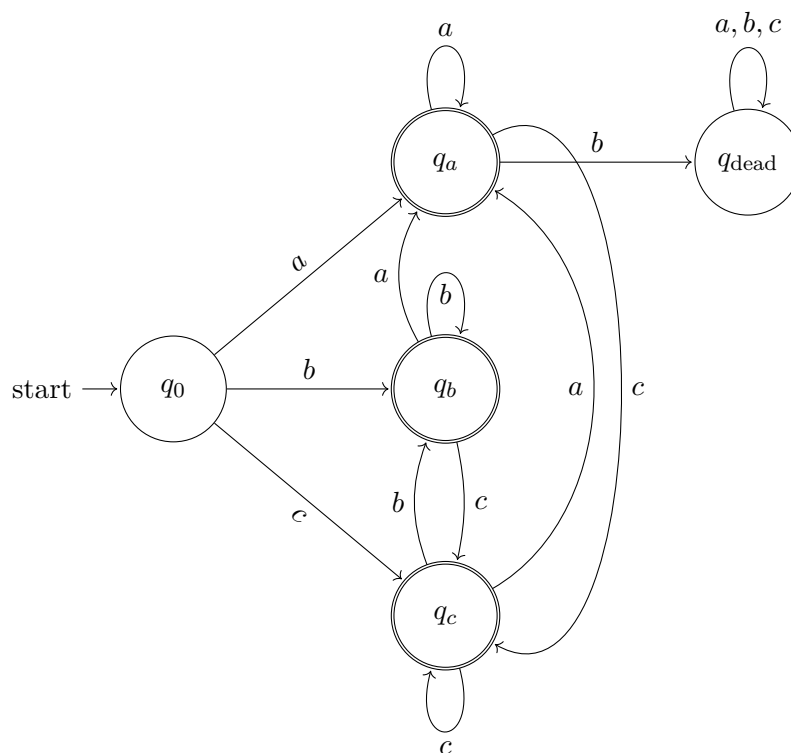
P.T.O

3. Tracking the current symbol

Another valid solution is to have three accepting states q_a, q_b, q_c to record which symbol was just read. Then, all inter-symbol transitions become legal except reading b immediately after a , which goes to the dead state.

$$Q = \{q_0, q_a, q_b, q_c, q_{\text{dead}}\}, \quad F = \{q_a, q_b, q_c\}$$

δ	a	b	c
$\rightarrow q_0$	q_a	q_b	q_c
q_a^*	q_a	q_{dead}	q_c
q_b^*	q_a	q_b	q_c
q_c^*	q_a	q_b	q_c
q_{dead}	q_{dead}	q_{dead}	q_{dead}



Note. There can be more equivalent DFAs which are right, however these are the only valid DFA descriptions found so far in the papers graded. For minor errors in the DFA construction which do not fundamentally alter the language they describe, marks have been awarded. However, even a small mistake which changes the language renders the entire solution invalid.

Part (b): Regular expression for L **[2 pts]****Method 1 — Using Arden's Lemma**

For $R = Q + RP$ with $\varepsilon \notin P$, the unique solution is $R = QP^*$.

$$q_0 = \varepsilon + q_0(b + c) + q_1c \quad (1)$$

$$q_1 = q_0a + q_1a \quad (2)$$

$$q_{\text{dead}} = q_1b + q_{\text{dead}}(a + b + c) \quad (3)$$

(q_{dead} is irrelevant to L and can be dropped.)

Solve Eq. 2: $q_1 = q_0a + q_1 \cdot a$. This is $R = Q + RP$ with $Q = q_0a$, $P = a$, so

$$q_1 = QP^* = q_0 a a^* = q_0 aa^*.$$

Substitute into Eq. 1:

$$q_0 = \varepsilon + q_0(b + c) + q_0 aa^*c = \varepsilon + q_0(b + c + aa^*c).$$

This is $R = Q + RP$ with $Q = \varepsilon$ and $P = b + c + aa^*c$, so by Arden's Lemma

$$q_0 = QP^* = (b + c + aa^*c)^*.$$

Simplifying P : $c + aa^*c = (\varepsilon + aa^*)c = a^*c$, so $P = b + a^*c$ and equivalently

$$q_0 = (b + a^*c)^*.$$

Back-substitute: $q_1 = q_0 aa^* = (b + a^*c)^* aa^*$.

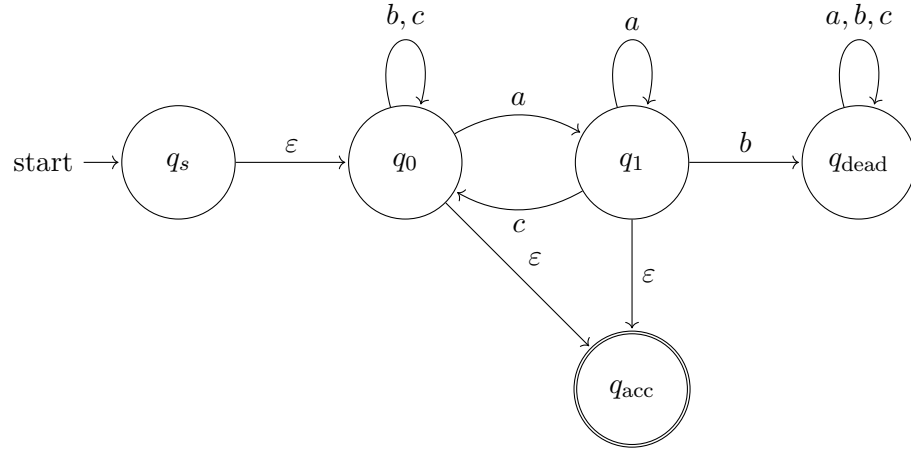
Accepting states are q_0, q_1 , so

$$L = q_0 + q_1 = (b + a^*c)^* + (b + a^*c)^* aa^* = (b + a^*c)^* (\varepsilon + aa^*) = (b + a^*c)^* a^*.$$

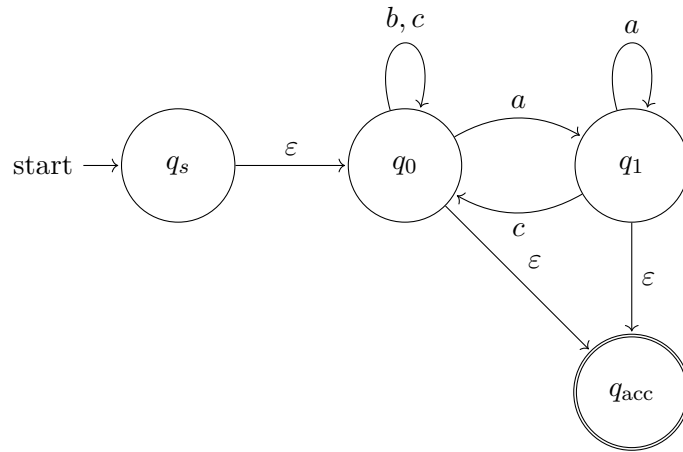
Grading:

- **Full marks** — for arriving at this regex or an equivalent valid one.
- **Partial (0.5)** — for mentioning Arden's Lemma OR writing down the correct state equations but with an incorrect solution.
- **Partial (0.5)** — for $\overline{(a + b + c)^* ab(a + b + c)^*}$ (the complement as a whole); partial credit has been awarded only because this is a technically valid regex but not what was expected.
- **0** otherwise. Do the spot checks given below for your answer.

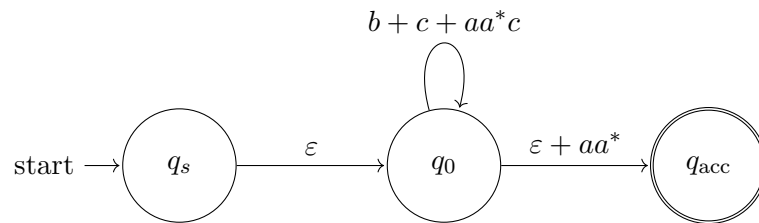
Method 2 — GNFA state elimination



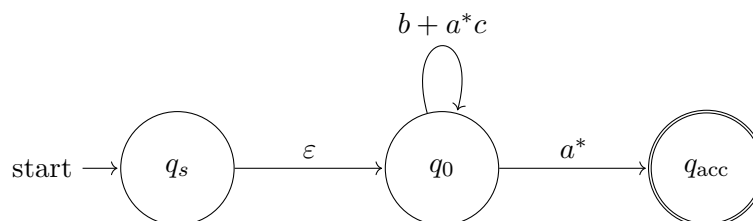
Step 1: eliminate q_{dead} (a non-accepting sink with no path to q_{acc}):



Step 2: eliminate q_1 . The path $q_0 \xrightarrow{a} q_1 \xrightarrow{a^*} q_1 \xrightarrow{c} q_0$ contributes a self-loop aa^*c on q_0 , and $q_0 \xrightarrow{a} q_1 \xrightarrow{a^*} q_{\text{acc}}$ contributes an edge aa^* to q_{acc} . The self-loop on q_0 becomes $b + c + aa^*c$:



Step 3: simplify the self-loop $b + c + aa^*c = b + a^*c$ and the exit edge $\varepsilon + aa^* = a^*$:



Eliminating q_0 gives $L = (b + a^*c)^*a^*$.

Grading: There are no partial marks for the first step of adding a new initial and final state, as it is trivial. The immediate next step of simplification directly leads to the valid regex and is necessary for full credit. As a result, there is no partial credit here.

Spot-checks

$ab, aab, cab, bab, aabc, cabc \notin L(r)$ (each contains ab);
 $\varepsilon, acb, a, b, c, bac, cba, aac, ba, ca, cc, aaa, acacb, bcacbbcc, \in L(r)$.

P.T.O

II. [3 Points] Construct a Context-Free Grammar (CFG) for the following language: the set of fully parenthesized arithmetic expressions over $\Sigma = \{0, 1, (,), +, \cdot\}$ that evaluate to 2 under the standard arithmetic interpretation.

(Note that the symbol $\cdot$ refers to multiplication)

For example, " $(0 + (1 \cdot 0) + 1 \cdot (1 + 0) + (1))$ " is a string that belongs to this language.

Solution:

Constructing a CFG for L

The main idea is to construct nonterminals according to the *value* of the expression they generate.

Intuition

Since every expression is formed using only 0, 1, addition, and multiplication, an expression whose value is 2 can only have the following possibilities at its outermost operation.

Addition. If an expression evaluates to 2 and its outermost operation is addition, then

$$2 = 0 + 2 = 2 + 0 = 1 + 1.$$

Thus, an expression evaluating to 2 can be formed from expressions evaluating to 0 and 2, or from two expressions evaluating to 1.

Multiplication. If an expression evaluates to 2 and its outermost operation is multiplication, then

$$2 = 1 \cdot 2 = 2 \cdot 1.$$

There are no other possibilities over nonnegative integers.

Therefore, it is sufficient to keep track of expressions evaluating to 0, 1, and 2.

We use the following nonterminals:

Z = expressions evaluating to 0,

O = expressions evaluating to 1,

T = expressions evaluating to 2,

and E for an arbitrary valid arithmetic expression.

The nonterminal E is needed because an expression of value 0 may have the form

$$(0 \cdot E)$$

or

$$(E \cdot 0),$$

where E can have *any* value.

CFG

Let $G = (V, \Sigma, P, S)$ where

$$V = \{S, E, Z, O, T\}, \quad \Sigma = \{0, 1, (,), +, \cdot\}$$

The productions are:

$$S \rightarrow T \tag{4}$$

$$E \rightarrow 0 \mid 1 \mid (E + E) \mid (E \cdot E) \tag{5}$$

$$Z \rightarrow 0 \mid (Z + Z) \mid (Z \cdot E) \mid (E \cdot Z) \tag{6}$$

$$O \rightarrow 1 \mid (Z + O) \mid (O + Z) \mid (O \cdot O) \tag{7}$$

$$T \rightarrow (Z + T) \mid (T + Z) \mid (O + O) \mid (O \cdot T) \mid (T \cdot O) \tag{8}$$

The start symbol is S .

The above solution is just one possible solution for this question. Any other valid solutions are also considered. The rubric for grading is as follows:

1. **[3 points]** For a complete grammar that describes the language given in the question.
2. **[2 points]** For a complete grammar that describes the language given in the question but has a single minor mistake, such as not handling a case.
3. **[1 point]** For a complete grammar that describes the language given in the question but has two minor mistakes.
4. **[0.5 points]** For a valid idea to construct the grammar, but did not actually describe the grammar.
5. **[0 points]** For any other invalid grammar.

2. [6 points] Let the rotational closure of a language A be defined as:

$$\text{RC}(A) = \{yx \mid xy \in A\}$$

- a. [3 points] Show that for any language A , we have $\text{RC}(A) = \text{RC}(\text{RC}(A))$.
b. [3 points] Show that the class of regular languages is closed under rotational closure.

Solution:

Part a: Prove $\text{RC}(A) = \text{RC}(\text{RC}(A))$ [3 points]

Marking Breakdown: The proof must show both directions of subset inclusion.

- Direction 1: $\text{RC}(A) \subseteq \text{RC}(\text{RC}(A))$ [1.5 marks]
- Direction 2: $\text{RC}(\text{RC}(A)) \subseteq \text{RC}(A)$ [1.5 marks]

Direction 1: Show $\text{RC}(A) \subseteq \text{RC}(\text{RC}(A))$ [1.5 marks]

Expected Proof: Let $w \in \text{RC}(A)$. We can write $w = w\epsilon$ (where $y = w$ and $x = \epsilon$). Since $w \in \text{RC}(A)$, taking the concatenation in the reverse order xy gives $\epsilon w = w$, which we already know is in $\text{RC}(A)$.

By the definition of Rotational Closure applied to the language $\text{RC}(A)$, since $xy \in \text{RC}(A)$, it must be that $yx \in \text{RC}(\text{RC}(A))$.

Substituting $y = w, x = \epsilon$, we get $w\epsilon = w \in \text{RC}(\text{RC}(A))$.

Step Marks for Direction 1:

- [1.0 mark] The insight to split the string using the empty string ϵ (i.e., $x = \epsilon, y = w$).
- [0.5 marks] Correctly concluding that $w \in \text{RC}(\text{RC}(A))$ based on the definition.
- *Partial Credit:* [0.5 marks] The idea of splitting the string is present, but mistakes are present in the execution.

Direction 2: Show $\text{RC}(\text{RC}(A)) \subseteq \text{RC}(A)$ [1.5 marks]

Expected Proof (Formal): Let $w \in \text{RC}(\text{RC}(A))$. By definition, $w = yx$ for some x, y such that $xy \in \text{RC}(A)$.

Since $xy \in \text{RC}(A)$, we can write $xy = vu$ for some u, v such that $uv \in A$.

We have $xy = vu$. There are two overlapping cases based on lengths:

1. **Case $|x| \leq |v|$:** Then $v = xz$ for some string z . We have $xy = xzu \implies y = zu$.
The original string in A is $uv = u(xz) \in A$.
Our string is $w = yx = (zu)x = zux$.
Let $p = ux$ and $q = z$. Then $pq = uxz \in A$. Thus, $qp = zux = w \in \text{RC}(A)$.

2. **Case $|x| > |v|$:** Then $x = vz$ for some string z . We have $xy = vzy = vu \implies u = zy$.
The original string in A is $uv = (zy)v \in A$.
Our string is $w = yx = y(vz) = yvz$.
Let $p = z$ and $q = yv$. Then $pq = zyv \in A$. Thus, $qp = yvz = w \in \text{RC}(A)$.

Alternative Accepted Proof (Conceptual/Intuitive): Let $w \in \text{RC}(\text{RC}(A))$. Any string in $\text{RC}(L)$ is simply a cyclic shift (rotation) of a string in L . Therefore, w is a cyclic shift of some string $w' \in \text{RC}(A)$, and w' is a cyclic shift of some string $w'' \in A$. Because the property of being a cyclic shift is transitive (a cyclic shift of a cyclic shift is just a cyclic shift of the original string), w is a cyclic shift of $w'' \in A$. Therefore, $w \in \text{RC}(A)$.

Step Marks for Direction 2:

- **[0.5 marks]** Correct setup: $w = yx$ where $xy \in \text{RC}(A)$ AND $xy = vu$ where $uv \in A$.
- **[1.0 mark]** Showing that transitivity holds (either via the rigorous split of indices z as shown in the formal proof, OR a clear, explicitly stated conceptual argument about transitivity of cyclic shifts).
- *Partial Credit:* **[0.75 marks]** An intuitive proof is provided, but no proof of transitivity of cyclic shifts or a constructive argument is present.

Part b: Show Regular Languages are closed under RC [3 points]

Students may approach this using Automata Properties (Finite Union of DFA paths) or NFA Construction. Both are valid.

Solution 1: Finite Union based on DFA States (Recommended & Most Elegant)

Let A be a regular language and $M = (Q, \Sigma, \delta, q_0, F)$ be its DFA.
For any state $q \in Q$, define two languages:

- $L_{q_0, q}$: The set of strings that take the DFA from the start state q_0 to state q .
- $L_{q, F}$: The set of strings that take the DFA from state q to some final state in F .

We can express the language A as the union over all states: $A = \bigcup_{q \in Q} (L_{q_0, q} L_{q, F})$.
A string $xy \in A$ means x drives the machine from q_0 to some state q , and y drives it from q to F . Thus, $x \in L_{q_0, q}$ and $y \in L_{q, F}$.

The rotational closure simply swaps these to yx . Therefore, we can express $\text{RC}(A)$ as:

$$\text{RC}(A) = \bigcup_{q \in Q} (L_{q, F} L_{q_0, q})$$

Since Q is finite, and regular languages are closed under concatenation and finite union, $\text{RC}(A)$ is regular.

Marking Scheme for Solution 1:

- **[1.0 mark]** Decomposing the strings into two halves based on intermediate DFA states.
- **[1.0 mark]** Correctly writing the expression for $\text{RC}(A)$ as $\bigcup(L_{q,F}L_{q_0,q})$.
- **[1.0 mark]** Explicitly mentioning closure properties (finite union and concatenation) to conclude it is regular.

Solution 2: NFA Construction

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA for A . We build an NFA M' for $\text{RC}(A)$ that “guesses” the state q_i where the string xy was split (x ends and y begins). M' operates in two phases:

1. **Phase 1 (Reading y):** Starts at the guessed state q_i and simulates M . It must reach a state in F .
2. **Phase 2 (Reading x):** Once in F , it takes a free ϵ -transition to q_0 , and simulates M on x . It accepts if it ends exactly at the originally guessed state q_i .

Formal Construction: $Q' = Q \times Q \times \{1, 2\} \cup \{s_{start}\}$

Start state: s_{start} with ϵ -transitions to $(q, q, 1)$ for all $q \in Q$. (The second coordinate remembers the guessed state).

Transitions:

- $\delta'((p, q, 1), a) = (\delta(p, a), q, 1)$ (Simulate y)
- $\delta'((p, q, 1), \epsilon) = (q_0, q, 2)$ if $p \in F$ (Jump from end of y to start of x)
- $\delta'((r, q, 2), a) = (\delta(r, a), q, 2)$ (Simulate x)

Final States: $F' = \{(q, q, 2) \mid q \in Q\}$.

Marking Scheme for Solution 2:

- **[1.0 mark]** The core idea: NFA guesses the split state, verifies y goes to F , and x goes from q_0 to the split state.
- **[1.0 mark]** Mechanism for Phase 1 to Phase 2 transition (e.g., ϵ -transition from final states back to q_0).
- **[1.0 mark]** Mechanism for verifying the NFA ends at the correctly guessed state (e.g., storing the guessed state in the tuple/state name and checking it at the end).
- *Partial Credit:* **[0.5 marks]** The split state is mentioned, but the rest of the construction is missing.
- *Note:* If the NFA is described purely in text without formal tuple notation, full points are awarded ONLY IF all mechanics (guessing, jumping, verifying) are completely unambiguous.

Common Mistakes

The following are some common mistakes that people have made. **please** read them before coming to the TA for queries. Any queries that are already explained in this section **will not be entertained**.

1. **Assuming the split is constant across nested closures:**

Assuming that if $xy \in A$, then $yx \in \text{RC}(A)$, and thus $xy \in \text{RC}(\text{RC}(A))$ by reusing the exact same split (x, y) . This assumes the rotation operation must split the string at the exact same boundary, which makes the proof trivial and incomplete. The split does not need to be constant.

Example: Let $A = \{abc\}$. A string in $\text{RC}(A)$ is bca (split as $u = a, v = bc \implies uv = abc \in A$, so $vu = bca \in \text{RC}(A)$). When rotating this string again to form a string in $\text{RC}(\text{RC}(A))$, we can choose an entirely different partition, such as $x = bc$ and $y = a$, yielding $yx = abc$. The second split does not have to align with the first.

2. **Claiming $\text{RC}(A) = A$:**

Asserting that the rotational closure of a language is always equal to the original language itself. In general, $A \subseteq \text{RC}(A)$ *Counterexample:* Let $A = \{abcd\}$. Then:

$$\text{RC}(A) = \{abcd, bcda, cdab, dabc\}$$

Here, $\text{RC}(A)$ contains 4 strings while A contains only 1. Thus, $\text{RC}(A) \neq A$, and $A \subsetneq \text{RC}(A)$.

3. **Equating cyclic rotation with string reversal:**

Treating the rotational closure $\text{RC}(A)$ as the language reversal A^R . Cyclic permutation preserves the internal sequential order of characters up to a wrap-around, whereas reversal completely mirrors the character sequence.

Counterexample: For $w = abcd$, its reversal is $w^R = dcba$. However:

$$\text{RC}(\{abcd\}) = \{abcd, bcda, cdab, dabc\}$$

Notice that $dcba \notin \text{RC}(\{abcd\})$.

4. **Assuming $xy \in A \implies yx \in A$:**

Assuming that strings in A are inherently commutative under concatenation or that A is already closed under rotation.

Counterexample: Let $A = \{ab\}$. Splitting ab into $x = a$ and $y = b$, we have $xy = ab \in A$. However, $yx = ba \notin A$. Swapping the partition guarantees $yx \in \text{RC}(A)$, not that $yx \in A$.

5. **Assuming that if $yx \in \text{RC}(A)$, then $xy \in A$:**

The definition states that $w \in \text{RC}(A)$ if *there exists* at least one decomposition $w = vu$ such that $uv \in A$. It does *not* state that for *any arbitrary* partition $w = yx$, swapping them gives a string in A .

Counterexample: Let $A = \{abcde\}$. Then:

$$\text{RC}(A) = \{abcde, bcdea, cdeab, deabc, eabcd\}$$

Take the string $w = bcdea \in \text{RC}(A)$. Choose the partition $y = b$ and $x = cdea$ (so $yx = bcdea$). Swapping the parts gives:

$$xy = cdeab$$

While $cdeab \in \text{RC}(A)$, $cdeab \notin A$ because A only contains $\{abcde\}$.

6. **Treating rotational closure as simple language concatenation:**

Claiming that $\text{RC}(A)$ is regular simply by asserting that “regular languages are closed under concatenation, so yx is formed by concatenating string y and string x .”

This argument is invalid because rotational closure requires x and y to originate from the *exact same* string $w = xy \in A$. Standard language concatenation (L_1L_2) forms a Cartesian cross-product, pairing every string in L_1 with every string in L_2 . Defining arbitrary prefix and suffix sets and concatenating them introduces strings that are formed by combining a suffix of one string with a prefix of an entirely different string, generating strings that do not belong to $\text{RC}(A)$. One must explicitly tie the split to matching internal DFA states to guarantee both parts belong to the same underlying string.

P.T.O

3. [4 points] Let $G = (V, E)$ be a directed graph with no self-loops.

A *closed walk* of length $n > 0$ in G is a sequence of vertices $v_1 v_2 \cdots v_{n+1}$ such that $v_i v_{i+1}$ is an edge in E for all $1 \leq i \leq n$, and $v_{n+1} = v_1$. Note that a closed walk can possibly repeat vertices multiple times.

A *cycle* is a closed walk that does not repeat vertices apart from the starting and ending vertex, which appears exactly twice. The *girth* of G is defined as the length of the shortest cycle in G .

For some $v \in V$, let L_v be the language over $\Sigma = V$ defined as

$$L_v = \{s \in V^* \mid s \text{ is a closed walk in } G \text{ starting and ending at } v \text{ of length at least } 2\}$$

Prove that if L_v is a non-empty regular language of pumping length $p > 0$, then G has girth strictly less than p .

Solution:

Proof. Observe that any closed walk of length n will necessarily contain a cycle within it of length $\leq n$ — either it is a cycle already, else there are some vertices which repeat, and we can always find a subsequence that is between two occurrences of a repeated vertex such that it has no other repeated vertices within it, giving us a cycle. Hence it suffices to prove that if L_v has pumping length p , then there is a string of length $< p$, as this will give us a closed walk of length $< p$ and hence a cycle of length $< p$. Clearly then the girth is $< p$.

Take the smallest string w in L_v (which must exist as L_v is non-empty), if it has length $< p$ we are done. Else, by the pumping lemma we can decompose it into $w = xyz$, $x, y, z \in V^*$, $|xy| \leq p$, $|y| \geq 1$ such that $xy^i z \in L_v$ for $i \geq 0$. By setting $i = 0$, we have that $xz \in L_v$, and as $|y| \geq 1$ we must have that $|xz| < |w|$, but this is a contradiction as w was assumed to be the shortest string in L_v , yet $xz \in L_v$. Hence the shortest string in L_v has length $< p$, which finishes off the proof. $\square$

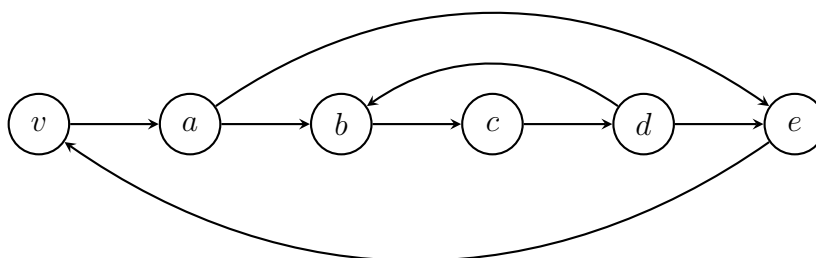
There may be alternate proofs not handled in the rubric below — they will be judged equally on an individual basis. We have tried to translate the structure of the rubric here to a rubric of that proof in these cases.

1. **0.5 points** for observing that a closed walk of length n contains a cycle of length $\leq n$ (does not need to be formally shown, merely observed).
2. **0.5 points** for observing further that the girth of G is strictly less than p if there is a closed walk of length p in L_v .
3. **0.5 points** for stating the pumping lemma in some capacity.
4. **1 point** for coming up with the following ideas, even if not properly executed.
 - Pumping down using the shortest string to get a contradiction.
 - Pumping down arbitrarily many times in succession from any string of length $\geq p$ to get a string of length $< p$.

5. **1.5 points** for a full proof using any of the ideas above to show the existence of a cycle of length $< p$. Points may be deducted per minor jump in logic that do not completely invalidate the proof attempt.

A lot of people tried pumping *up* instead — the issue with this method is you will only get that the girth is of length $\leq p$ via this method (atleast none of the solutions have managed to obtain $< p$) These attempts have received a maximum of **2** marks depending on how complete the attempt was: the first 3 points in the above rubric stay common, and an additional **0.5** for correctly stating the idea and getting a conclusion.

One recurring mistake people made was assuming that y may be a cycle/closed walk - this **need not** be true. For example observe that in the below graph for $x = va$, $z = ev$, $y = bcd$ can be pumped (up or down) but y itself does not form a cycle (it would need to be $y = bcd b$). Hence you get that the length of the shortest cycle is $\leq p$ in the best case.



Another common mistake was assuming that $|x| > 0$, this again need not be true - in the above example, remove v, a and set $b = v$, add an edge from b to e and back. Then $y = bcd$ can be pumped multiple times as long as it is suffixed with $z = beb$.

It is true that you can still reason in both the above cases too, but the assumptions as stated do not hold. If you have not considered the counterexamples separately you will not receive marks.

Another separate solution tried to use the DFA corresponding to L_v — most solutions were not convincing enough to justify the length of the loop, since they did not specify *which* DFA, and without this information it is impossible to imply something about G . Some solutions used the NFA constructed from G instead, the marking scheme in this case is the same as the one for the previous proof since these are basically equivalent forms of the problem.

Another popular method was a proof by contradiction, that somehow the pumping lemma would fail if the girth of G was atleast p - unfortunately a proof attempt of this kind fails for multiple reasons. The first is that the witness that girth of G is $\geq p$ may not be a string in L_v , so you cannot use it and pump down automatically - you must reason this out first. You can then try a similar solution as in the rubric to pump down to obtain a contradiction somehow - I am unsure whether a proof strategy of this kind would work but no one's solution got this far anyways. Thus these solutions only received partial marks depending on completeness, for a maximum of **2** marks, with grading similar to the

pumping up method above. If you have shown the steps **clearly** and I have not awarded points, please see me (Pratyush) in the paper showing.

I have tried to go through the merits of each solution and assess whether the idea was feasible - even an incorrect solution would receive at least **0.5** points for the idea if it was somewhat feasible. If you *genuinely* feel this was the case for your idea and I have not given points, please do see me (Pratyush) in the paper showing, but do be sure about this before approaching.

P.T.O

4. [2 points] Prove, using the pumping lemma for CFLs, that the language L defined over $\Sigma = \{a, b\}$ as

$$L = \{a^n b^m \mid m = n^2, n \geq 0\}$$

is not a context-free language.

Solution:

Assume for contradiction that L is a context-free language. Let $p \geq 1$ be its pumping length. Choose the string $s = a^p b^{p^2}$. Since $s \in L$ and $|s| \geq p$, the pumping lemma applies. We can decompose $s = uvwxy$ such that $|vwx| \leq p$, $|vx| \geq 1$, and $uv^t wx^t y \in L$ for all $t \geq 0$.

Because $|vwx| \leq p$, the substring vwx can only be in one of three places: entirely in the a 's, entirely in the b 's, or crossing the boundary between them. We show that all cases break the $m = n^2$ rule.

Case 1: vwx consists only of a 's. Pump down by setting $t = 0$. The resulting string has fewer a 's (some $n' < p$) but still has exactly p^2 b 's. Since $(n')^2 < p^2$, the pumped string is no longer in L .

Case 2: vwx consists only of b 's. Pumping down ($t = 0$) reduces the number of b 's while leaving exactly p a 's. The new number of b 's is now strictly less than p^2 , so the string is not in L .

For the above cases, pumping up and proving the inequality also works.

Case 3: vwx crosses the boundary between a 's and b 's. Pump up by setting $t = 2$. In doing this, we add at most p characters overall. If we added at least one a , our new string has $n' \geq p + 1$ a 's. For this to stay in L , we would need at least $(p + 1)^2 = p^2 + 2p + 1$ b 's. However, because we only added at most p characters in total, we can't possibly add the $2p + 1$ extra b 's required. On the other hand, if we added no a 's, we only added b 's, which ruins the exact p^2 match since the a 's didn't change.

In all cases, the pumped string leaves L . This contradicts the pumping lemma, meaning L cannot be context-free.

Marking Scheme (Total: 2 Marks)

All solutions were graded accordingly but here's the general breakdown for the most common one:

- **0.5 marks: Pumping Lemma setup and string choice**

We awarded 0.5 for correctly stating and applying the Pumping Lemma for context-free languages and choosing a valid string in L whose length is at least the pumping length p , such as $s = a^p b^{p^2}$.

If the Pumping Lemma is fundamentally stated incorrectly, 0 marks were given for this component.

- **0.75 marks: Mixed a/b pumping case**

For correctly handling the case where the pumped portions involve both types of characters, meaning the pumping affects both the a -part and the b -part of the string.

The student should show that, for some valid pumping choice, usually pumping up or down, the resulting numbers of a 's and b 's no longer satisfy the required relation $m = n^2$.

Any correct argument establishing the contradiction is acceptable. It does not need to follow the exact algebra or positional split used in the reference solution.

- **0.75 marks: Pumping only one type of character**

We divided this component as follows:

- **0.25 marks** for identifying the general idea that if the pumped portions affect only one type of character, one side of the relation changes while the other stays fixed.
- **0.25 marks** for correctly handling the case where only a 's are pumped. The number of a 's changes while the number of b 's stays fixed, so the relation $m = n^2$ is violated.
- **0.25 marks** for correctly handling the case where only b 's are pumped. The number of b 's changes while the number of a 's stays fixed, so the relation $m = n^2$ is violated.

I would treat the a -only and b -only cases separately because they involve different parts of the string and need different arguments.

- **Case coverage (any valid split is acceptable):**

The answer does not need to follow the exact case division shown in the reference solution. I would accept any complete case analysis, whether it is organized around the positions of v and x , the characters they contain, or changes in the numbers of a 's and b 's, as long as all possible configurations are covered and the reasoning in each case is correct.

- **Partial credit for missing or incorrect cases:**

If one particular case, or an equivalent subcase, is missing, mishandled, or not justified, I would deduct marks only from the corresponding component above. I would not invalidate the rest of an otherwise correct case analysis.

- **Deductions:**

I would deduct up to 0.25 marks for minor algebraic mistakes, off-by-one errors, notation issues, or small logical gaps that do not affect the central contradiction.